FILE: /app/frontend/src/App.js

```
import "@/App.css";
import { useEffect } from "react";
import { BrowserRouter, Navigate, Route, Routes } from "react-router-dom";
import { ThemeProvider } from "next-themes";
import { Toaster } from "@/components/ui/sonner";
import { TopNav } from "@/components/TopNav";
import { useStudyData } from "@/hooks/useStudyData";
import DashboardPage from "@/pages/DashboardPage";
import PlannerPage from "@/pages/PlannerPage";
import TasksPage from "@/pages/TasksPage";
import FocusPage from "@/pages/FocusPage";
import DeadlinesPage from "@/pages/DeadlinesPage";
import AnalyticsPage from "@/pages/AnalyticsPage";

const PlannerRoutes = ({
  generatingPlan,
  submittingTask,
  submittingDeadline,
  tasks,
  deadlines,
  reminders,
  plans,
  focusSessions,
  analytics,
  addTask,
  toggleTask,
  deleteTask,
  updateTaskDetails,
  addDeadline,
  toggleDeadline,
  deleteDeadline,
  updateDeadlineDetails,
  logFocusSession,
  generatePlan,
}) => {
  return (
    <Routes>
      <Route
        path="/dashboard"
        element={<DashboardPage tasks={tasks} deadlines={deadlines} analytics={analytics} plans={plans} />}
      />
      <Route
        path="/planner"
        element={<PlannerPage generatingPlan={generatingPlan} latestPlan={plans[0]} onGeneratePlan={generatePlan} />}
      />
      <Route
        path="/tasks"
        element={
          <TasksPage
            submittingTask={submittingTask}
            tasks={tasks}
            onAddTask={addTask}
            onToggleTask={toggleTask}
            onDeleteTask={deleteTask}
            onUpdateTask={updateTaskDetails}
          />
        }
      />
      <Route path="/focus" element={<FocusPage focusSessions={focusSessions} onLogSession={logFocusSession} />} />
      <Route
        path="/deadlines"
        element={
          <DeadlinesPage
            submittingDeadline={submittingDeadline}
            deadlines={deadlines}
            reminders={reminders}
            onAddDeadline={addDeadline}
            onToggleDeadline={toggleDeadline}
            onDeleteDeadline={deleteDeadline}
            onUpdateDeadline={updateDeadlineDetails}
          />
        }
      />
      <Route path="/analytics" element={<AnalyticsPage analytics={analytics} />} />
      <Route path="*" element={<Navigate to="/dashboard" replace />} />
    </Routes>
  );
};

function App() {
  const {
    loading,
    generatingPlan,
```

```
    submittingTask,
    submittingDeadline,
    tasks,
    deadlines,
    reminders,
    plans,
    focusSessions,
    analytics,
    addTask,
    toggleTask,
    deleteTask,
    updateTaskDetails,
    addDeadline,
    toggleDeadline,
    deleteDeadline,
    updateDeadlineDetails,
    logFocusSession,
    generatePlan,
  } = useStudyData();

  useEffect(() => {
    const hideInjectedBadge = () => {
      const candidates = document.querySelectorAll("a, div, span");
      candidates.forEach((node) => {
        if ((node.textContent || "").includes("Made with Emergent")) {
          node.style.setProperty("display", "none", "important");
          node.style.setProperty("visibility", "hidden", "important");
          node.style.setProperty("opacity", "0", "important");
          node.style.setProperty("pointer-events", "none", "important");
        }
      });
    };

    hideInjectedBadge();
    const observer = new MutationObserver(() => hideInjectedBadge());
    observer.observe(document.body, { childList: true, subtree: true });

    return () => observer.disconnect();
  }, []);

  return (
    <ThemeProvider attribute="class" defaultTheme="light" enableSystem={false}>
      <BrowserRouter>
        <div className="planner-bg" data-testid="study-planner-app-shell">
          <TopNav />
          <main className="page-shell">
            {loading ? (
              <div
                className="brutal-card bg-white p-8 text-base md:text-lg"
                data-testid="global-loading-card"
              >
                Loading your study space...
              </div>
            ) : (
              <PlannerRoutes
                generatingPlan={generatingPlan}
                submittingTask={submittingTask}
                submittingDeadline={submittingDeadline}
                tasks={tasks}
                deadlines={deadlines}
                reminders={reminders}
                plans={plans}
                focusSessions={focusSessions}
                analytics={analytics}
                addTask={addTask}
                toggleTask={toggleTask}
                deleteTask={deleteTask}
                updateTaskDetails={updateTaskDetails}
                addDeadline={addDeadline}
                toggleDeadline={toggleDeadline}
                deleteDeadline={deleteDeadline}
                updateDeadlineDetails={updateDeadlineDetails}
                logFocusSession={logFocusSession}
                generatePlan={generatePlan}
              />
            )}
          </main>
          <Toaster data-testid="toast-root" position="top-right" richColors closeButton />
        </div>
      </BrowserRouter>
    </ThemeProvider>
  );
}

```

```
export default App;
```

FILE: /app/frontend/src/App.css

```
.planner-bg {
  min-height: 100vh;
  background-color: #fafaf9;
  background-image:
    radial-gradient(circle at 20% 10%, rgba(191, 219, 254, 0.5), transparent 35%),
    radial-gradient(circle at 80% 90%, rgba(187, 247, 208, 0.55), transparent 35%),
    radial-gradient(circle at 60% 40%, rgba(233, 213, 255, 0.45), transparent 40%);
}

.page-shell {
  padding: 2rem 1rem 7rem;
}

@media (min-width: 768px) {
  .page-shell {
    padding: 2.5rem 2rem 8rem;
  }
}

.brutal-card {
  border: 2px solid #18181b;
  border-radius: 0;
  box-shadow: 4px 4px 0 0 #18181b;
  transition: transform 180ms ease, box-shadow 180ms ease;
}

.brutal-card:hover {
  transform: translate(2px, 2px);
  box-shadow: 0 0 0 0 #18181b;
}

.brutal-input {
  border: 0;
  border-bottom: 2px solid #18181b;
  border-radius: 0;
  padding: 0.5rem 0;
  background: transparent;
  transition: border-width 120ms ease;
}

.brutal-input:focus {
  outline: none;
  border-bottom-width: 4px;
}

.timer-font {
  font-family: "JetBrains Mono", monospace;
}

.card-appear {
  animation: cardFadeUp 350ms ease forwards;
}

@keyframes cardFadeUp {
  from {
    opacity: 0;
    transform: translateY(12px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

FILE: /app/frontend/src/index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import "@fontsource/syne/700.css";
import "@fontsource/outfit/400.css";
import "@fontsource/outfit/500.css";
import "@fontsource/jetbrains-mono/500.css";
import "@/index.css";
import App from "@/App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

FILE: /app/frontend/src/index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  margin: 0;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

code {
  font-family: "JetBrains Mono", monospace;
}

@layer base {
  :root {
    --background: 48 33% 98%;
    --foreground: 240 10% 10%;
    --card: 0 0% 100%;
    --card-foreground: 240 10% 10%;
    --popover: 0 0% 100%;
    --popover-foreground: 240 10% 10%;
    --primary: 240 10% 10%;
    --primary-foreground: 0 0% 98%;
    --secondary: 240 17% 96%;
    --secondary-foreground: 240 10% 10%;
    --muted: 220 14% 96%;
    --muted-foreground: 240 5% 40%;
    --accent: 213 88% 87%;
    --accent-foreground: 240 10% 10%;
    --destructive: 0 76% 62%;
    --destructive-foreground: 0 0% 98%;
    --border: 240 10% 10%;
    --input: 240 10% 10%;
    --ring: 240 10% 10%;
    --chart-1: 6 92% 81%;
    --chart-2: 210 88% 87%;
    --chart-3: 136 71% 84%;
    --chart-4: 31 90% 84%;
    --chart-5: 273 100% 92%;
    --radius: 0rem;
  }

  .dark {
    --background: 240 12% 7%;
    --foreground: 0 0% 96%;
    --card: 240 10% 10%;
    --card-foreground: 0 0% 96%;
    --popover: 240 10% 10%;
    --popover-foreground: 0 0% 96%;
    --primary: 0 0% 96%;
    --primary-foreground: 240 10% 10%;
    --secondary: 240 8% 20%;
    --secondary-foreground: 0 0% 96%;
    --muted: 240 8% 20%;
    --muted-foreground: 240 5% 70%;
    --accent: 240 8% 20%;
    --accent-foreground: 0 0% 96%;
    --destructive: 0 62% 30%;
    --destructive-foreground: 0 0% 96%;
    --border: 0 0% 96%;
    --input: 240 8% 20%;
    --ring: 0 0% 96%;
  }
}

@layer base {
  * {
    @apply border-border;
  }

  body {
    @apply bg-background text-foreground;
    font-family: "Outfit", sans-serif;
  }

  h1,
  h2,
  h3,
  h4 {
    font-family: "Syne", sans-serif;
    letter-spacing: -0.02em;
  }
```

```
}

@layer base {
  [data-debug-wrapper="true"] {
    display: contents !important;
  }

  [data-debug-wrapper="true"] > * {
    margin-left: inherit;
    margin-right: inherit;
    margin-top: inherit;
    margin-bottom: inherit;
    padding-left: inherit;
    padding-right: inherit;
    padding-top: inherit;
    padding-bottom: inherit;
    column-gap: inherit;
    row-gap: inherit;
    gap: inherit;
    border-left-width: inherit;
    border-right-width: inherit;
    border-top-width: inherit;
    border-bottom-width: inherit;
    border-left-style: inherit;
    border-right-style: inherit;
    border-top-style: inherit;
    border-bottom-style: inherit;
    border-left-color: inherit;
    border-right-color: inherit;
    border-top-color: inherit;
    border-bottom-color: inherit;
  }
}

#emergent-badge {
  display: none !important;
  visibility: hidden !important;
  opacity: 0 !important;
  pointer-events: none !important;
}
```

FILE: /app/frontend/src/api/studyApi.js

```
import axios from "axios";

const API_BASE = `${process.env.REACT_APP_BACKEND_URL}/api`;

const api = axios.create({
  baseURL: API_BASE,
  headers: {
    "Content-Type": "application/json",
  },
});

export const getTasks = async () => (await api.get("/tasks")).data;
export const createTask = async (payload) => (await api.post("/tasks", payload)).data;
export const updateTask = async (taskId, payload) => (await api.put(`/tasks/${taskId}`, payload)).data;
export const removeTask = async (taskId) => (await api.delete(`/tasks/${taskId}`)).data;

export const getDeadlines = async () => (await api.get("/deadlines")).data;
export const createDeadline = async (payload) => (await api.post("/deadlines", payload)).data;
export const updateDeadline = async (deadlineId, payload) => (await api.put(`/deadlines/${deadlineId}`, payload)).data;
export const removeDeadline = async (deadlineId) => (await api.delete(`/deadlines/${deadlineId}`)).data;
export const getReminders = async () => (await api.get("/reminders")).data;

export const getPlans = async () => (await api.get("/plans")).data;
export const generateStudyPlan = async (payload) => (await api.post("/plans/generate", payload)).data;

export const getFocusSessions = async () => (await api.get("/focus-sessions")).data;
export const createFocusSession = async (payload) => (await api.post("/focus-sessions", payload)).data;

export const getAnalyticsSummary = async () => (await api.get("/analytics/summary")).data;
```

FILE: /app/frontend/src/hooks/useStudyData.js

```
import { useCallback, useEffect, useState } from "react";
import { toast } from "@/components/ui/sonner";
import {
  createDeadline,
  createFocusSession,
  createTask,
  generateStudyPlan,
  getAnalyticsSummary,
  getDeadlines,
  getFocusSessions,
  getPlans,
```

```
  getReminders,
  getTasks,
  removeDeadline,
  removeTask,
  updateDeadline,
  updateTask,
} from "@/api/studyApi";

const EMPTY_ANALYTICS = {
  tasks_total: 0,
  tasks_completed: 0,
  deadlines_upcoming: 0,
  focus_minutes_total: 0,
  focus_sessions_count: 0,
  productivity_score: 0,
  subject_breakdown: [],
};

export const useStudyData = () => {
  const [loading, setLoading] = useState(true);
  const [generatingPlan, setGeneratingPlan] = useState(false);
  const [submittingTask, setSubmittingTask] = useState(false);
  const [submittingDeadline, setSubmittingDeadline] = useState(false);
  const [tasks, setTasks] = useState([]);
  const [deadlines, setDeadlines] = useState([]);
  const [reminders, setReminders] = useState([]);
  const [plans, setPlans] = useState([]);
  const [focusSessions, setFocusSessions] = useState([]);
  const [analytics, setAnalytics] = useState(EMPTY_ANALYTICS);

  const loadAllData = useCallback(async () => {
    try {
      const [taskData, deadlineData, reminderData, planData, sessionData, analyticsData] = await Promise.all([
        getTasks(),
        getDeadlines(),
        getReminders(),
        getPlans(),
        getFocusSessions(),
        getAnalyticsSummary(),
      ]);
      setTasks(taskData);
      setDeadlines(deadlineData);
      setReminders(reminderData);
      setPlans(planData);
      setFocusSessions(sessionData);
      setAnalytics(analyticsData);
    } catch {
      toast.error("Unable to load planner data. Please refresh.");
    } finally {
      setLoading(false);
    }
  }, []);

  const mutateAndRefresh = async ({ action, successMessage, errorMessage, onStart, onDone }) => {
    try {
      if (onStart) {
        onStart();
      }
      await action();
      toast.success(successMessage);
      await loadAllData();
    } catch {
      toast.error(errorMessage);
    } finally {
      if (onDone) {
        onDone();
      }
    }
  };

  useEffect(() => {
    void loadAllData();
  }, [loadAllData]);

  const addTask = async (payload) => {
    await mutateAndRefresh({
      action: () => createTask(payload),
      successMessage: "Task added.",
      errorMessage: "Could not add task.",
      onStart: () => setSubmittingTask(true),
      onDone: () => setSubmittingTask(false),
    });
  };

```

```
  const toggleTask = async (task) => {
    await mutateAndRefresh({
      action: () => updateTask(task.id, { completed: !task.completed }),
      successMessage: task.completed ? "Task reopened." : "Task completed.",
      errorMessage: "Could not update task.",
    });
  };

  const deleteTask = async (taskId) => {
    await mutateAndRefresh({
      action: () => removeTask(taskId),
      successMessage: "Task deleted.",
      errorMessage: "Could not delete task.",
    });
  };

  const updateTaskDetails = async (taskId, payload) => {
    await mutateAndRefresh({
      action: () => updateTask(taskId, payload),
      successMessage: "Task updated.",
      errorMessage: "Could not update task.",
    });
  };

  const addDeadline = async (payload) => {
    await mutateAndRefresh({
      action: () => createDeadline(payload),
      successMessage: "Deadline added.",
      errorMessage: "Could not add deadline.",
      onStart: () => setSubmittingDeadline(true),
      onDone: () => setSubmittingDeadline(false),
    });
  };

  const toggleDeadline = async (deadline) => {
    await mutateAndRefresh({
      action: () => updateDeadline(deadline.id, { completed: !deadline.completed }),
      successMessage: deadline.completed ? "Deadline moved back to active." : "Deadline marked complete.",
      errorMessage: "Could not update deadline.",
    });
  };

  const deleteDeadline = async (deadlineId) => {
    await mutateAndRefresh({
      action: () => removeDeadline(deadlineId),
      successMessage: "Deadline removed.",
      errorMessage: "Could not delete deadline.",
    });
  };

  const updateDeadlineDetails = async (deadlineId, payload) => {
    await mutateAndRefresh({
      action: () => updateDeadline(deadlineId, payload),
      successMessage: "Deadline updated.",
      errorMessage: "Could not update deadline.",
    });
  };

  const logFocusSession = async (durationMinutes) => {
    await mutateAndRefresh({
      action: () =>
        createFocusSession({
          duration_minutes: durationMinutes,
          completed: true,
          started_at: new Date().toISOString(),
        }),
      successMessage: "Focus session logged.",
      errorMessage: "Unable to log focus session.",
    });
  };

  const generatePlan = async (payload) => {
    await mutateAndRefresh({
      action: () => generateStudyPlan(payload),
      successMessage: "AI schedule generated.",
      errorMessage: "AI plan generation failed.",
      onStart: () => setGeneratingPlan(true),
      onDone: () => setGeneratingPlan(false),
    });
  };

  return {
    loading,
    generatingPlan,
```

```
    submittingTask,
    submittingDeadline,
    tasks,
    deadlines,
    reminders,
    plans,
    focusSessions,
    analytics,
    addTask,
    toggleTask,
    deleteTask,
    updateTaskDetails,
    addDeadline,
    toggleDeadline,
    deleteDeadline,
    updateDeadlineDetails,
    logFocusSession,
    generatePlan,
  };
};
```

FILE: /app/frontend/src/components/TopNav.jsx

```
import { Link, NavLink } from "react-router-dom";
import { BarChart3, Brain, CalendarClock, CheckSquare, House, Timer } from "lucide-react";

const routes = [
  { to: "/dashboard", label: "Dashboard", icon: House },
  { to: "/planner", label: "AI Planner", icon: Brain },
  { to: "/tasks", label: "Tasks", icon: CheckSquare },
  { to: "/focus", label: "Focus", icon: Timer },
  { to: "/deadlines", label: "Deadlines", icon: CalendarClock },
  { to: "/analytics", label: "Analytics", icon: BarChart3 },
];

export const TopNav = () => {
  return (
    <header className="sticky top-0 z-40 bg-[#fafaf9]/90 px-4 pb-4 pt-6 backdrop-blur md:px-8"
data-testid="top-nav-wrapper">
      <div className="mb-5 flex flex-wrap items-center justify-between gap-4">
        <Link
          to="/dashboard"
          className="flex items-center gap-3 text-[#18181b] transition-colors hover:text-zinc-700"
          data-testid="brand-home-link"
        >
          <span className="inline-flex h-10 w-10 items-center justify-center border-2 border-[#18181b] bg-[#bbf7d0] text-lg
font-bold">
            SP
          </span>
          <div>
            <h1 className="text-2xl sm:text-3xl">Study Planner AI</h1>
            <p className="text-sm text-zinc-600" data-testid="brand-subtitle">
              Plan smarter, focus deeper, finish earlier.
            </p>
          </div>
        </Link>
      </div>

      <nav className="flex flex-wrap gap-3" data-testid="top-nav-links">
        {routes.map(({ to, label, icon: Icon }) => (
          <NavLink
            key={to}
            to={to}
            className={({ isActive }) =>
              [
                "inline-flex items-center gap-2 rounded-full border-2 border-[#18181b] px-4 py-2 text-sm font-medium
transition-transform",
                isActive ? "bg-[#18181b] text-white" : "bg-white text-[#18181b] hover:-translate-y-1",
              ].join(" ")
            }
            data-testid={`nav-link-${label.toLowerCase().replace(/\s+/g, "-")}`}
          >
            <Icon size={16} strokeWidth={2} />
            {label}
          </NavLink>
        ))}
      </nav>
    </header>
  );
};
```

FILE: /app/frontend/src/components/BrutalCard.jsx

```
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";

export const BrutalCard = ({ title, subtitle, className = "", cardTestId, children }) => {
```

```
  return (
    <Card className={`brutal-card bg-white ${className}`} data-testid={cardTestId}>
      {(title || subtitle) && (
        <CardHeader className="space-y-2 p-6">
          {title ? (
            <CardTitle className="text-xl text-[#18181b]" data-testid={`${cardTestId}-title`}>
              {title}
            </CardTitle>
          ) : null}
          {subtitle ? (
            <p className="text-sm text-zinc-700" data-testid={`${cardTestId}-subtitle`}>
              {subtitle}
            </p>
          ) : null}
        </CardHeader>
      )}
      <CardContent className="p-6 pt-0">{children}</CardContent>
    </Card>
  );
};
```

FILE: /app/frontend/src/components/DatePickerField.jsx

```
import { CalendarIcon } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Calendar } from "@/components/ui/calendar";
import { Popover, PopoverContent, PopoverTrigger } from "@/components/ui/popover";

const toDateObject = (value) => {
  if (!value) {
    return undefined;
  }
  const parsed = new Date(value);
  return Number.isNaN(parsed.getTime()) ? undefined : parsed;
};

const toDisplayValue = (value) => {
  const dateValue = toDateObject(value);
  if (!dateValue) {
    return "Pick date";
  }
  return dateValue.toLocaleDateString();
};

export const DatePickerField = ({ id, label, value, onChange, required = false, testIdPrefix }) => {
  return (
    <div className="space-y-2" data-testid={`${testIdPrefix}-wrapper`}>
      <label htmlFor={id} className="text-sm font-medium" data-testid={`${testIdPrefix}-label`}>
        {label}
      </label>
      <Popover>
        <PopoverTrigger asChild>
          <Button
            id={id}
            type="button"
            className="w-full justify-between rounded-none border-2 border-[#18181b] bg-white text-[#18181b]
hover:bg-[#18181b] hover:text-white"
            data-testid={`${testIdPrefix}-button`}
          >
            <span data-testid={`${testIdPrefix}-value`}>{toDisplayValue(value)}</span>
            <CalendarIcon size={16} />
          </Button>
        </PopoverTrigger>
        <PopoverContent className="w-auto border-2 border-[#18181b] bg-white p-0" align="start"
data-testid={`${testIdPrefix}-popover`}>
          <Calendar
            mode="single"
            selected={toDateObject(value)}
            onSelect={(date) => {
              if (!date) {
                if (!required) {
                  onChange("");
                }
                return;
              }
              onChange(date.toISOString().slice(0, 10));
            }}
            initialFocus
            data-testid={`${testIdPrefix}-calendar`}
          />
        </PopoverContent>
      </Popover>
    </div>
  );
};
```

FILE: /app/frontend/src/pages/DashboardPage.jsx

```
import { Link } from "react-router-dom";
import { motion } from "framer-motion";
import { ArrowRight, CheckCircle2, Clock3, Sparkles } from "lucide-react";
import { Button } from "@/components/ui/button";
import { BrutalCard } from "@/components/BrutalCard";

const DASHBOARD_MOTION_PROPS = {
  initial: { opacity: 0, y: 20 },
  animate: { opacity: 1, y: 0 },
  transition: { duration: 0.35 },
};

const toReadableDate = (value) => {
  if (!value) {
    return "No date";
  }
  const parsed = new Date(value);
  if (Number.isNaN(parsed.getTime())) {
    return value;
  }
  return parsed.toLocaleDateString();
};

export default function DashboardPage({ tasks, deadlines, analytics, plans }) {
  const pendingTasks = tasks.filter((task) => !task.completed).slice(0, 4);
  const upcomingDeadlines = deadlines.filter((deadline) => !deadline.completed).slice(0, 5);
  const latestPlan = plans[0];

  return (
    <section className="space-y-8" data-testid="dashboard-page-root">
      <motion.div className="grid grid-cols-1 gap-8 lg:grid-cols-3" {...DASHBOARD_MOTION_PROPS}>
        <div className="brutal-card card-appear overflow-hidden border-2 border-[#18181b] bg-white lg:col-span-2"
data-testid="dashboard-hero-card">
          <div className="grid gap-0 md:grid-cols-2">
            <div className="space-y-5 p-7">
              <p className="text-sm font-medium uppercase tracking-wider text-zinc-600" data-testid="dashboard-hero-label">
                Your daily sanctuary
              </p>
              <h2 className="text-4xl sm:text-5xl lg:text-6xl" data-testid="dashboard-hero-title">
                Calm plan. <br /> Bold progress.
              </h2>
              <p className="text-sm md:text-base" data-testid="dashboard-hero-description">
                Build your AI timetable, execute focused blocks, and stay ahead of every deadline.
              </p>
              <Button
                asChild
                className="rounded-full border-2 border-[#18181b] bg-[#18181b] px-7 py-6 font-bold text-white
transition-transform hover:scale-105"
                data-testid="dashboard-open-planner-button"
              >
                <Link to="/planner">
                  Open AI Planner <ArrowRight className="ml-1" size={16} />
                </Link>
              </Button>
            </div>
            <div className="relative aspect-[4/3] md:aspect-auto" data-testid="dashboard-hero-image-wrap">
              <img
                src="https://images.unsplash.com/photo-1650661926447-9efb2610f64c?crop=entropy&cs=srgb&fm=jpg&ixid=M3w4NjA1N
zV8MHwxfHNlYXJjaHwxfHxhZXN0aGV0aWMlMjBzdHVkeSUyMGRlc2slMjBzZXR1cCUyMHN0dWRlbnR8ZW58MHx8fHwxNzczNjQyMDcwfDA&ixlib=rb-4.1.0&q=85"
                alt="Study desk with planner"
                className="h-full w-full object-cover object-center"
                data-testid="dashboard-hero-image"
              />
            </div>
          </div>
        </div>

        <BrutalCard
          className="bg-[#fecaca]"
          title="Today at a glance"
          subtitle="Core metrics"
          cardTestId="dashboard-summary-card"
        >
          <div className="space-y-4 text-sm md:text-base">
            <div className="flex items-center justify-between" data-testid="dashboard-metric-pending-tasks-row">
              <span>Pending tasks</span>
              <strong data-testid="dashboard-metric-pending-tasks-value">{pendingTasks.length}</strong>
            </div>
            <div className="flex items-center justify-between" data-testid="dashboard-metric-focus-minutes-row">
              <span>Focus minutes</span>
              <strong data-testid="dashboard-metric-focus-minutes-value">{analytics.focus_minutes_total}</strong>
            </div>
            <div className="flex items-center justify-between" data-testid="dashboard-metric-score-row">
```

```
  81|               <span>Productivity score</span>
  82|               <strong data-testid="dashboard-metric-score-value">{analytics.productivity_score}%</strong>
  83|             </div>
  84|           </div>
  85|         </BrutalCard>
  86|       </motion.div>
  87|
  88|       <div className="grid grid-cols-1 gap-6 md:grid-cols-4" data-testid="dashboard-bento-grid">
  89|         <BrutalCard className="bg-[#fecaca] md:col-span-1" title="Focus" subtitle="Pomodoro readiness"
cardTestId="dashboard-focus-card">
  90|           <div className="space-y-3">
  91|             <p className="text-sm" data-testid="dashboard-focus-text">Start a 25-minute sprint and log your session.</p>
  92|             <Button
  93|               asChild
  94|               className="rounded-full border-2 border-[#18181b] bg-white text-[#18181b] hover:bg-[#18181b] hover:text-white"
  95|               data-testid="dashboard-go-focus-button"
  96|             >
  97|               <Link to="/focus">Go to Focus</Link>
  98|             </Button>
  99|           </div>
 100|         </BrutalCard>
 101|
 102|         <BrutalCard className="bg-white md:col-span-2" title="Today's Plan" subtitle="From your latest AI schedule"
cardTestId="dashboard-latest-plan-card">
 103|           <div className="space-y-3" data-testid="dashboard-latest-plan-content">
 104|             {latestPlan ? (
 105|               latestPlan.generated_schedule.slice(0, 2).map((day, index) => (
 106|                 <div className="border-b-2 border-[#18181b] pb-3" key={`${day.day}-${index}`}
data-testid={`dashboard-plan-day-${index}`}>
 107|                   <p className="text-sm font-semibold" data-testid={`dashboard-plan-day-${index}-name`}>{day.day}</p>
 108|                   <p className="text-sm" data-testid={`dashboard-plan-day-${index}-topics`}>{day.topics.join(", ")}</p>
 109|                 </div>
 110|               ))
 111|             ) : (
 112|               <p className="text-sm" data-testid="dashboard-empty-plan-message">No AI plan yet. Generate one in AI
Planner.</p>
 113|             )}
 114|           </div>
 115|         </BrutalCard>
 116|
 117|         <BrutalCard className="bg-[#bbf7d0] md:col-span-1" title="Progress" subtitle="Task completion"
cardTestId="dashboard-progress-card">
 118|           <div className="space-y-2">
 119|             <p className="text-sm" data-testid="dashboard-progress-count">
 120|               {analytics.tasks_completed} / {analytics.tasks_total} tasks done
 121|             </p>
 122|             <div className="h-3 w-full border-2 border-[#18181b] bg-white" data-testid="dashboard-progress-bar-track">
 123|               <div
 124|                 className="h-full bg-[#18181b]"
 125|                 style={{ width: `${analytics.tasks_total ? (analytics.tasks_completed / analytics.tasks_total) * 100 : 0}%`
}}
 126|                 data-testid="dashboard-progress-bar-fill"
 127|               />
 128|             </div>
 129|           </div>
 130|         </BrutalCard>
 131|       </div>
 132|
 133|       <BrutalCard
 134|         className="bg-[#bfdbfe]"
 135|         title="Upcoming Deadlines"
 136|         subtitle="Stay ahead of critical dates"
 137|         cardTestId="dashboard-deadline-card"
 138|       >
 139|         <div className="space-y-3" data-testid="dashboard-deadline-list">
 140|           {upcomingDeadlines.length === 0 ? (
 141|             <p className="text-sm" data-testid="dashboard-deadline-empty">No upcoming deadlines right now.</p>
 142|           ) : (
 143|             upcomingDeadlines.map((deadline, index) => (
 144|               <div
 145|                 className="flex flex-wrap items-center justify-between gap-3 border-b-2 border-[#18181b] pb-3"
 146|                 key={deadline.id}
 147|                 data-testid={`dashboard-deadline-item-${index}`}
 148|               >
 149|                 <div>
 150|                   <p className="font-medium" data-testid={`dashboard-deadline-item-${index}-title`}>{deadline.title}</p>
 151|                   <p className="text-sm" data-testid={`dashboard-deadline-item-${index}-date`}>
 152|                     Due {toReadableDate(deadline.due_date)}
 153|                   </p>
 154|                 </div>
 155|                 <div className="inline-flex items-center gap-2 text-sm"
data-testid={`dashboard-deadline-item-${index}-status`}>
 156|                   {deadline.completed ? <CheckCircle2 size={16} /> : <Clock3 size={16} />}
 157|                   {deadline.completed ? "Done" : "Active"}
```

```
158|                </div>
159|              </div>
160|            ))
161|          )}
162|        </div>
163|        <div className="mt-5 flex flex-wrap gap-3">
164|          <Button
165|            asChild
166|            className="rounded-full border-2 border-[#18181b] bg-[#18181b] px-7 py-6 font-bold text-white
transition-transform hover:scale-105"
167|            data-testid="dashboard-open-deadlines-button"
168|          >
169|            <Link to="/deadlines">Manage Deadlines</Link>
170|          </Button>
171|          <Button
172|            asChild
173|            variant="outline"
174|            className="rounded-full border-2 border-[#18181b] bg-white px-7 py-6 font-bold text-[#18181b] hover:bg-[#18181b]
hover:text-white"
175|            data-testid="dashboard-open-analytics-button"
176|          >
177|            <Link to="/analytics">
178|              View Analytics <Sparkles className="ml-1" size={16} />
179|            </Link>
180|          </Button>
181|        </div>
182|      </BrutalCard>
183|    </section>
184|  );
185| }
```

FILE: /app/frontend/src/pages/PlannerPage.jsx

```
  1| import { useState } from "react";
  2| import { motion } from "framer-motion";
  3| import { WandSparkles } from "lucide-react";
  4| import { Button } from "@/components/ui/button";
  5| import { Input } from "@/components/ui/input";
  6| import { Textarea } from "@/components/ui/textarea";
  7| import { BrutalCard } from "@/components/BrutalCard";
  8|
  9| const DEFAULT_AVAILABLE_HOURS = 3;
 10| const DEFAULT_STUDY_DAYS = 7;
 11| const PAGE_MOTION_PROPS = {
 12|   initial: { opacity: 0, y: 20 },
 13|   animate: { opacity: 1, y: 0 },
 14|   transition: { duration: 0.35 },
 15| };
 16|
 17| export default function PlannerPage({ generatingPlan, latestPlan, onGeneratePlan }) {
 18|   const [goals, setGoals] = useState("");
 19|   const [priorities, setPriorities] = useState("");
 20|   const [availableHours, setAvailableHours] = useState(DEFAULT_AVAILABLE_HOURS);
 21|   const [studyDays, setStudyDays] = useState(DEFAULT_STUDY_DAYS);
 22|
 23|   const handleSubmit = async (event) => {
 24|     event.preventDefault();
 25|     await onGeneratePlan({
 26|       goals,
 27|       priorities,
 28|       available_hours_per_day: Number(availableHours),
 29|       study_days: Number(studyDays),
 30|     });
 31|   };
 32|
 33|   return (
 34|     <section className="space-y-8" data-testid="planner-page-root">
 35|       <motion.div {...PAGE_MOTION_PROPS} className="grid grid-cols-1 gap-6 xl:grid-cols-2">
 36|         <BrutalCard
 37|           className="bg-[#e9d5ff]"
 38|           title="AI Study Timetable Generator"
 39|           subtitle="Describe your goal and get a practical, day-by-day plan."
 40|           cardTestId="planner-form-card"
 41|         >
 42|           <form className="space-y-6" onSubmit={handleSubmit} data-testid="planner-form">
 43|             <div className="space-y-2">
 44|               <label htmlFor="planner-goals" className="text-sm font-medium" data-testid="planner-goals-label">
 45|                 Study Goals
 46|               </label>
 47|               <Textarea
 48|                 id="planner-goals"
 49|                 value={goals}
 50|                 onChange={(event) => setGoals(event.target.value)}
 51|                 placeholder="Example: Math finals, organic chemistry revision, and essay writing practice"
 52|                 className="min-h-24 border-2 border-[#18181b] rounded-none"
```

```
  53|                required
  54|                data-testid="planner-goals-input"
  55|              />
  56|            </div>
  57|
  58|            <div className="space-y-2">
  59|              <label htmlFor="planner-priorities" className="text-sm font-medium" data-testid="planner-priorities-label">
  60|                Priorities
  61|              </label>
  62|              <Input
  63|                id="planner-priorities"
  64|                value={priorities}
  65|                onChange={(event) => setPriorities(event.target.value)}
  66|                placeholder="Weak topics first, daily revision, practice questions"
  67|                className="border-2 border-[#18181b] rounded-none"
  68|                data-testid="planner-priorities-input"
  69|              />
  70|            </div>
  71|
  72|            <div className="grid grid-cols-1 gap-5 md:grid-cols-2">
  73|              <div className="space-y-2">
  74|                <label htmlFor="planner-hours" className="text-sm font-medium" data-testid="planner-hours-label">
  75|                  Hours available per day
  76|                </label>
  77|                <Input
  78|                  id="planner-hours"
  79|                  type="number"
  80|                  min="1"
  81|                  max="12"
  82|                  value={availableHours}
  83|                  onChange={(event) => setAvailableHours(event.target.value)}
  84|                  className="border-2 border-[#18181b] rounded-none"
  85|                  data-testid="planner-hours-input"
  86|                />
  87|              </div>
  88|              <div className="space-y-2">
  89|                <label htmlFor="planner-days" className="text-sm font-medium" data-testid="planner-days-label">
  90|                  Study days
  91|                </label>
  92|                <Input
  93|                  id="planner-days"
  94|                  type="number"
  95|                  min="3"
  96|                  max="30"
  97|                  value={studyDays}
  98|                  onChange={(event) => setStudyDays(event.target.value)}
  99|                  className="border-2 border-[#18181b] rounded-none"
 100|                  data-testid="planner-days-input"
 101|                />
 102|              </div>
 103|            </div>
 104|
 105|            <Button
 106|              type="submit"
 107|              className="w-full rounded-full border-2 border-[#18181b] bg-[#18181b] py-6 text-base font-bold text-white
transition-transform hover:scale-105"
 108|              disabled={generatingPlan || !goals.trim()}
 109|              data-testid="planner-generate-button"
 110|            >
 111|              <WandSparkles className="mr-2" size={16} />
 112|              {generatingPlan ? "Generating..." : "Generate AI Study Plan"}
 113|            </Button>
 114|          </form>
 115|        </BrutalCard>
 116|
 117|        <BrutalCard
 118|          className="bg-white"
 119|          title="Latest Generated Plan"
 120|          subtitle="Use this as your daily execution roadmap."
 121|          cardTestId="planner-result-card"
 122|        >
 123|          {latestPlan ? (
 124|            <div className="space-y-4" data-testid="planner-generated-plan-wrap">
 125|              <p className="text-sm md:text-base" data-testid="planner-generated-plan-summary">{latestPlan.summary}</p>
 126|              {latestPlan.generated_schedule.map((day, index) => (
 127|                <div className="border-b-2 border-[#18181b] pb-3" key={`${day.day}-${index}`}
data-testid={`planner-day-item-${index}`}>
 128|                  <p className="font-medium" data-testid={`planner-day-item-${index}-name`}>{day.day}</p>
 129|                  <p className="text-sm" data-testid={`planner-day-item-${index}-topics`}>
 130|                    Topics: {day.topics.join(", ")}
 131|                  </p>
 132|                  <p className="text-sm" data-testid={`planner-day-item-${index}-minutes`}>
 133|                    Focus: {day.focus_minutes} min
 134|                  </p>
```

```
                  <p className="text-sm" data-testid={`planner-day-item-${index}-notes`}>{day.notes}</p>
                </div>
              ))}
            </div>
          ) : (
            <p className="text-sm" data-testid="planner-empty-message">
              No plan yet. Fill the form and generate your first AI timetable.
            </p>
          )}
        </BrutalCard>
      </motion.div>
    </section>
  );
}
```

FILE: /app/frontend/src/pages/TasksPage.jsx

```
import { useState } from "react";
import { motion } from "framer-motion";
import { Check, Pencil, Plus, Save, Trash2, X } from "lucide-react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { BrutalCard } from "@/components/BrutalCard";
import { DatePickerField } from "@/components/DatePickerField";

const DEFAULT_ESTIMATED_MINUTES = 45;
const ACTION_ICON_SIZE = 15;
const TASK_STAGGER_DELAY = 0.05;
const TASK_ITEM_VARIANTS = {
  hidden: { opacity: 0, y: 8 },
  visible: (index) => ({
    opacity: 1,
    y: 0,
    transition: { delay: index * TASK_STAGGER_DELAY },
  }),
};

const toDisplayDate = (value) => {
  if (!value) {
    return "No due date";
  }
  const parsed = new Date(value);
  return Number.isNaN(parsed.getTime()) ? value : parsed.toLocaleDateString();
};

const toDateInput = (value) => {
  if (!value) {
    return "";
  }
  const parsed = new Date(value);
  if (Number.isNaN(parsed.getTime())) {
    return "";
  }
  return parsed.toISOString().slice(0, 10);
};

export default function TasksPage({
  submittingTask,
  tasks,
  onAddTask,
  onToggleTask,
  onDeleteTask,
  onUpdateTask,
}) {
  const [title, setTitle] = useState("");
  const [subject, setSubject] = useState("General");
  const [priority, setPriority] = useState("medium");
  const [dueDate, setDueDate] = useState("");
  const [estimatedMinutes, setEstimatedMinutes] = useState(DEFAULT_ESTIMATED_MINUTES);
  const [editingTaskId, setEditingTaskId] = useState("");
  const [taskDraft, setTaskDraft] = useState({
    title: "",
    subject: "General",
    priority: "medium",
    dueDate: "",
    estimatedMinutes: DEFAULT_ESTIMATED_MINUTES,
  });

  const submitTask = async (event) => {
    event.preventDefault();
    await onAddTask({
      title,
      subject,
      priority,
      due_date: dueDate ? new Date(dueDate).toISOString() : null,
```

```
      estimated_minutes: Number(estimatedMinutes),
      completed: false,
    });
    setTitle("");
    setSubject("General");
    setPriority("medium");
    setDueDate("");
    setEstimatedMinutes(DEFAULT_ESTIMATED_MINUTES);
  };

  const startTaskEdit = (task) => {
    setEditingTaskId(task.id);
    setTaskDraft({
      title: task.title,
      subject: task.subject,
      priority: task.priority,
      dueDate: toDateInput(task.due_date),
      estimatedMinutes: task.estimated_minutes,
    });
  };

  const cancelTaskEdit = () => {
    setEditingTaskId("");
  };

  const saveTaskEdit = async () => {
    if (!editingTaskId || !taskDraft.title.trim()) {
      return;
    }
    await onUpdateTask(editingTaskId, {
      title: taskDraft.title,
      subject: taskDraft.subject,
      priority: taskDraft.priority,
      due_date: taskDraft.dueDate ? new Date(taskDraft.dueDate).toISOString() : null,
      estimated_minutes: Number(taskDraft.estimatedMinutes),
    });
    setEditingTaskId("");
  };

  return (
    <section className="space-y-8" data-testid="tasks-page-root">
      <div className="grid grid-cols-1 gap-6 xl:grid-cols-2">
        <BrutalCard
          className="bg-[#bfdbfe]"
          title="Create Daily Task"
          subtitle="Quickly add what needs to get done."
          cardTestId="task-create-card"
        >
          <form className="space-y-5" onSubmit={submitTask} data-testid="task-create-form">
            <div className="space-y-2">
              <label htmlFor="task-title" className="text-sm font-medium" data-testid="task-title-label">Task title</label>
              <Input
                id="task-title"
                value={title}
                onChange={(event) => setTitle(event.target.value)}
                required
                placeholder="Solve chapter 5 practice questions"
                className="border-2 border-[#18181b] rounded-none"
                data-testid="task-title-input"
              />
            </div>

            <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
              <div className="space-y-2">
                <label htmlFor="task-subject" className="text-sm font-medium"
data-testid="task-subject-label">Subject</label>
                <Input
                  id="task-subject"
                  value={subject}
                  onChange={(event) => setSubject(event.target.value)}
                  className="border-2 border-[#18181b] rounded-none"
                  data-testid="task-subject-input"
                />
              </div>
              <div className="space-y-2">
                <label htmlFor="task-priority" className="text-sm font-medium"
data-testid="task-priority-label">Priority</label>
                <select
                  id="task-priority"
                  value={priority}
                  onChange={(event) => setPriority(event.target.value)}
                  className="h-10 w-full border-2 border-[#18181b] bg-white px-3 text-sm"
                  data-testid="task-priority-select"
                >
```

```
151|                    <option value="low">Low</option>
152|                    <option value="medium">Medium</option>
153|                    <option value="high">High</option>
154|                  </select>
155|                </div>
156|              </div>
157|
158|              <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
159|                <DatePickerField
160|                  id="task-due-date"
161|                  label="Due date"
162|                  value={dueDate}
163|                  onChange={setDueDate}
164|                  testIdPrefix="task-due-date"
165|                />
166|                <div className="space-y-2">
167|                  <label htmlFor="task-estimate" className="text-sm font-medium" data-testid="task-estimate-label">
168|                    Estimated minutes
169|                  </label>
170|                  <Input
171|                    id="task-estimate"
172|                    type="number"
173|                    min="5"
174|                    max="240"
175|                    value={estimatedMinutes}
176|                    onChange={(event) => setEstimatedMinutes(event.target.value)}
177|                    className="border-2 border-[#18181b] rounded-none"
178|                    data-testid="task-estimate-input"
179|                  />
180|                </div>
181|              </div>
182|
183|              <Button
184|                type="submit"
185|                disabled={submittingTask || !title.trim()}
186|                className="w-full rounded-full border-2 border-[#18181b] bg-[#18181b] py-6 text-base font-bold text-white
transition-transform hover:scale-105"
187|                data-testid="task-submit-button"
188|              >
189|                <Plus className="mr-2" size={16} />
190|                {submittingTask ? "Adding..." : "Add Task"}
191|              </Button>
192|            </form>
193|          </BrutalCard>
194|
195|          <BrutalCard
196|            className="bg-white"
197|            title="Task Queue"
198|            subtitle="Check off tasks as you complete them."
199|            cardTestId="task-list-card"
200|          >
201|            <div className="space-y-3" data-testid="task-list-wrap">
202|              {tasks.length === 0 ? (
203|                <p className="text-sm" data-testid="task-empty-message">No tasks yet. Add your first task.</p>
204|              ) : (
205|                tasks.map((task, index) => (
206|                  <motion.div
207|                    key={task.id}
208|                    custom={index}
209|                    variants={TASK_ITEM_VARIANTS}
210|                    initial="hidden"
211|                    animate="visible"
212|                    className="flex flex-wrap items-center justify-between gap-3 border-b-2 border-[#18181b] pb-3"
213|                    data-testid={`task-item-${index}`}
214|                  >
215|                    {editingTaskId === task.id ? (
216|                      <div className="w-full space-y-4" data-testid={`task-item-${index}-edit-wrap`}>
217|                        <div className="grid grid-cols-1 gap-3 md:grid-cols-2">
218|                          <div className="space-y-2">
219|                            <label className="text-sm font-medium" data-testid={`task-item-${index}-edit-title-label`}>
220|                              Title
221|                            </label>
222|                            <Input
223|                              value={taskDraft.title}
224|                              onChange={(event) => setTaskDraft((prev) => ({ ...prev, title: event.target.value }))}
225|                              className="border-2 border-[#18181b] rounded-none"
226|                              data-testid={`task-item-${index}-edit-title-input`}
227|                            />
228|                          </div>
229|                          <div className="space-y-2">
230|                            <label className="text-sm font-medium" data-testid={`task-item-${index}-edit-subject-label`}>
231|                              Subject
232|                            </label>
233|                            <Input
```

```
                          value={taskDraft.subject}
                          onChange={(event) => setTaskDraft((prev) => ({ ...prev, subject: event.target.value }))}
                          className="border-2 border-[#18181b] rounded-none"
                          data-testid={`task-item-${index}-edit-subject-input`}
                        />
                      </div>
                    </div>
                    <div className="grid grid-cols-1 gap-3 md:grid-cols-3">
                      <div className="space-y-2">
                        <label className="text-sm font-medium" data-testid={`task-item-${index}-edit-priority-label`}>
                          Priority
                        </label>
                        <select
                          value={taskDraft.priority}
                          onChange={(event) => setTaskDraft((prev) => ({ ...prev, priority: event.target.value }))}
                          className="h-10 w-full border-2 border-[#18181b] bg-white px-3 text-sm"
                          data-testid={`task-item-${index}-edit-priority-select`}
                        >
                          <option value="low">Low</option>
                          <option value="medium">Medium</option>
                          <option value="high">High</option>
                        </select>
                      </div>
                      <DatePickerField
                        id={`task-edit-due-date-${index}`}
                        label="Due date"
                        value={taskDraft.dueDate}
                        onChange={(value) => setTaskDraft((prev) => ({ ...prev, dueDate: value }))}
                        testIdPrefix={`task-item-${index}-edit-due-date`}
                      />
                      <div className="space-y-2">
                        <label className="text-sm font-medium" data-testid={`task-item-${index}-edit-estimate-label`}>
                          Estimated minutes
                        </label>
                        <Input
                          type="number"
                          min="5"
                          max="240"
                          value={taskDraft.estimatedMinutes}
                          onChange={(event) =>
                            setTaskDraft((prev) => ({ ...prev, estimatedMinutes: event.target.value }))
                          }
                          className="border-2 border-[#18181b] rounded-none"
                          data-testid={`task-item-${index}-edit-estimate-input`}
                        />
                      </div>
                    </div>
                    <div className="flex flex-wrap gap-2">
                      <Button
                        type="button"
                        onClick={saveTaskEdit}
                        className="rounded-full border-2 border-[#18181b] bg-[#18181b] text-white hover:scale-105"
                        data-testid={`task-item-${index}-edit-save-button`}
                      >
                        <Save size={ACTION_ICON_SIZE} className="mr-1" />
                        Save
                      </Button>
                      <Button
                        type="button"
                        onClick={cancelTaskEdit}
                        className="rounded-full border-2 border-[#18181b] bg-white text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                        data-testid={`task-item-${index}-edit-cancel-button`}
                      >
                        <X size={ACTION_ICON_SIZE} className="mr-1" />
                        Cancel
                      </Button>
                    </div>
                  </div>
                ) : (
                  <>
                    <div className="space-y-1">
                      <p
                        className={`font-medium ${task.completed ? "line-through text-zinc-500" : ""}`}
                        data-testid={`task-item-${index}-title`}
                      >
                        {task.title}
                      </p>
                      <p className="text-sm" data-testid={`task-item-${index}-meta`}>
                        {task.subject} • {task.priority} • {toDisplayDate(task.due_date)} • {task.estimated_minutes} min
                      </p>
                    </div>
                    <div className="flex gap-2">
                      <Button
```

```
                          type="button"
                          onClick={() => startTaskEdit(task)}
                          className="rounded-full border-2 border-[#18181b] bg-[#e9d5ff] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                          data-testid={`task-item-${index}-edit-button`}
                        >
                          <Pencil size={ACTION_ICON_SIZE} />
                        </Button>
                        <Button
                          type="button"
                          onClick={() => onToggleTask(task)}
                          className="rounded-full border-2 border-[#18181b] bg-[#bbf7d0] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                          data-testid={`task-item-${index}-toggle-button`}
                        >
                          <Check size={ACTION_ICON_SIZE} />
                        </Button>
                        <Button
                          type="button"
                          onClick={() => onDeleteTask(task.id)}
                          className="rounded-full border-2 border-[#18181b] bg-[#fecaca] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                          data-testid={`task-item-${index}-delete-button`}
                        >
                          <Trash2 size={ACTION_ICON_SIZE} />
                        </Button>
                      </div>
                    </>
                  )}
                </motion.div>
              ))
            )}
          </div>
        </BrutalCard>
      </div>
    </section>
  );
}
```

FILE: /app/frontend/src/pages/FocusPage.jsx

```
import { useEffect, useMemo, useState } from "react";
import { motion } from "framer-motion";
import { Pause, Play, RotateCcw, Save } from "lucide-react";
import { Button } from "@/components/ui/button";
import { BrutalCard } from "@/components/BrutalCard";
import { toast } from "@/components/ui/sonner";

const formatSeconds = (value) => {
  const minutes = Math.floor(value / 60);
  const seconds = value % 60;
  return `${String(minutes).padStart(2, "0")}:${String(seconds).padStart(2, "0")}`;
};

const formatSessionDate = (value) => {
  const parsed = new Date(value);
  return Number.isNaN(parsed.getTime()) ? value : parsed.toLocaleString();
};

const DEFAULT_FOCUS_MINUTES = 25;
const FOCUS_MODE_OPTIONS = [25, 50];
const CONTROL_ICON_SIZE = 16;
const STAGGER_DELAY = 0.05;
const HISTORY_ITEM_VARIANTS = {
  hidden: { opacity: 0, y: 8 },
  visible: (index) => ({
    opacity: 1,
    y: 0,
    transition: { delay: index * STAGGER_DELAY },
  }),
};

export default function FocusPage({ focusSessions, onLogSession }) {
  const [modeMinutes, setModeMinutes] = useState(DEFAULT_FOCUS_MINUTES);
  const [secondsLeft, setSecondsLeft] = useState(DEFAULT_FOCUS_MINUTES * 60);
  const [isRunning, setIsRunning] = useState(false);

  const finishedMinutes = useMemo(() => Math.round((modeMinutes * 60 - secondsLeft) / 60), [modeMinutes, secondsLeft]);

  useEffect(() => {
    if (!isRunning) {
      return undefined;
    }

    const interval = setInterval(() => {
```

```
      setSecondsLeft((previous) => {
        if (previous <= 1) {
          setIsRunning(false);
          toast.success("Pomodoro complete! Session logged.");
          onLogSession(modeMinutes);
          return 0;
        }
        return previous - 1;
      });
    }, 1000);

    return () => clearInterval(interval);
  }, [isRunning, modeMinutes, onLogSession]);

  const switchMode = (minutes) => {
    setModeMinutes(minutes);
    setSecondsLeft(minutes * 60);
    setIsRunning(false);
  };

  const resetTimer = () => {
    setSecondsLeft(modeMinutes * 60);
    setIsRunning(false);
  };

  const saveCurrentProgress = async () => {
    if (!finishedMinutes) {
      toast.info("Start the timer to log focus minutes.");
      return;
    }
    await onLogSession(finishedMinutes);
    resetTimer();
  };

  return (
    <section className="space-y-8" data-testid="focus-page-root">
      <div className="grid grid-cols-1 gap-6 xl:grid-cols-2">
        <BrutalCard
          className="bg-[#fecaca]"
          title="Pomodoro Focus Timer"
          subtitle="Stay intense, then recover quickly."
          cardTestId="focus-timer-card"
        >
          <div className="space-y-6" data-testid="focus-timer-wrap">
            <div className="flex flex-wrap gap-3" data-testid="focus-mode-buttons-wrap">
              {FOCUS_MODE_OPTIONS.map((minutes) => (
                <Button
                  key={minutes}
                  type="button"
                  onClick={() => switchMode(minutes)}
                  className={`rounded-full border-2 border-[#18181b] ${
                    modeMinutes === minutes ? "bg-[#18181b] text-white" : "bg-white text-[#18181b]"
                  } hover:bg-[#18181b] hover:text-white`}
                  data-testid={`focus-mode-${minutes}-button`}
                >
                  {minutes} min mode
                </Button>
              ))}
            </div>

            <p className="timer-font text-6xl font-semibold" data-testid="focus-timer-display">
              {formatSeconds(secondsLeft)}
            </p>

            <div className="flex flex-wrap gap-3" data-testid="focus-controls-wrap">
              <Button
                type="button"
                onClick={() => setIsRunning((previous) => !previous)}
                className="rounded-full border-2 border-[#18181b] bg-[#18181b] text-white hover:scale-105"
                data-testid="focus-toggle-button"
              >
                {isRunning ? <Pause size={CONTROL_ICON_SIZE} className="mr-2" /> : <Play size={CONTROL_ICON_SIZE}
className="mr-2" />}
                {isRunning ? "Pause" : "Start"}
              </Button>
              <Button
                type="button"
                onClick={resetTimer}
                className="rounded-full border-2 border-[#18181b] bg-white text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                data-testid="focus-reset-button"
              >
                <RotateCcw size={CONTROL_ICON_SIZE} className="mr-2" />
                Reset
```

```
 127|              </Button>
 128|              <Button
 129|                type="button"
 130|                onClick={saveCurrentProgress}
 131|                className="rounded-full border-2 border-[#18181b] bg-[#bbf7d0] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
 132|                data-testid="focus-save-progress-button"
 133|              >
 134|                <Save size={CONTROL_ICON_SIZE} className="mr-2" />
 135|                Log {finishedMinutes || 0} min
 136|              </Button>
 137|            </div>
 138|          </div>
 139|        </BrutalCard>
 140|
 141|        <BrutalCard className="bg-white" title="Recent Focus Sessions" subtitle="Your concentration history"
cardTestId="focus-history-card">
 142|          <div className="space-y-3" data-testid="focus-history-list">
 143|            {focusSessions.length === 0 ? (
 144|              <p className="text-sm" data-testid="focus-history-empty">No sessions logged yet.</p>
 145|            ) : (
 146|              focusSessions.slice(0, 8).map((session, index) => (
 147|                <motion.div
 148|                  key={session.id}
 149|                  custom={index}
 150|                  variants={HISTORY_ITEM_VARIANTS}
 151|                  initial="hidden"
 152|                  animate="visible"
 153|                  className="flex items-center justify-between border-b-2 border-[#18181b] pb-3"
 154|                  data-testid={`focus-history-item-${index}`}
 155|                >
 156|                  <p className="text-sm" data-testid={`focus-history-item-${index}-date`}>
 157|                    {formatSessionDate(session.started_at)}
 158|                  </p>
 159|                  <p className="text-sm font-medium" data-testid={`focus-history-item-${index}-minutes`}>
 160|                    {session.duration_minutes} min
 161|                  </p>
 162|                </motion.div>
 163|              ))
 164|            )}
 165|          </div>
 166|        </BrutalCard>
 167|      </div>
 168|    </section>
 169|  );
 170| }
```

FILE: /app/frontend/src/pages/DeadlinesPage.jsx

```
   1| import { useState } from "react";
   2| import { motion } from "framer-motion";
   3| import { Bell, Check, Pencil, Save, Trash2, X } from "lucide-react";
   4| import { Button } from "@/components/ui/button";
   5| import { Input } from "@/components/ui/input";
   6| import { Textarea } from "@/components/ui/textarea";
   7| import { BrutalCard } from "@/components/BrutalCard";
   8| import { DatePickerField } from "@/components/DatePickerField";
   9|
  10| const formatDate = (value) => {
  11|   const parsed = new Date(value);
  12|   return Number.isNaN(parsed.getTime()) ? value : parsed.toLocaleDateString();
  13| };
  14|
  15| const urgencyColor = (daysLeft) => {
  16|   if (daysLeft < 0) {
  17|     return "bg-[#fecaca]";
  18|   }
  19|   if (daysLeft <= 2) {
  20|     return "bg-[#fed7aa]";
  21|   }
  22|   return "bg-[#bbf7d0]";
  23| };
  24|
  25| const ACTION_ICON_SIZE = 15;
  26| const DEADLINE_STAGGER_DELAY = 0.05;
  27| const DEADLINE_ITEM_VARIANTS = {
  28|   hidden: { opacity: 0, y: 8 },
  29|   visible: (index) => ({
  30|     opacity: 1,
  31|     y: 0,
  32|     transition: { delay: index * DEADLINE_STAGGER_DELAY },
  33|   }),
  34| };
  35|
  36| const toDateInput = (value) => {
```

```
  if (!value) {
    return "";
  }
  const parsed = new Date(value);
  if (Number.isNaN(parsed.getTime())) {
    return "";
  }
  return parsed.toISOString().slice(0, 10);
};

export default function DeadlinesPage({
  submittingDeadline,
  deadlines,
  reminders,
  onAddDeadline,
  onToggleDeadline,
  onDeleteDeadline,
  onUpdateDeadline,
}) {
  const [title, setTitle] = useState("");
  const [subject, setSubject] = useState("General");
  const [dueDate, setDueDate] = useState("");
  const [reminderDate, setReminderDate] = useState("");
  const [notes, setNotes] = useState("");
  const [editingDeadlineId, setEditingDeadlineId] = useState("");
  const [deadlineDraft, setDeadlineDraft] = useState({
    title: "",
    subject: "General",
    dueDate: "",
    reminderDate: "",
    notes: "",
  });

  const submitDeadline = async (event) => {
    event.preventDefault();
    await onAddDeadline({
      title,
      subject,
      due_date: new Date(dueDate).toISOString(),
      reminder_date: reminderDate ? new Date(reminderDate).toISOString() : null,
      notes,
      completed: false,
    });
    setTitle("");
    setSubject("General");
    setDueDate("");
    setReminderDate("");
    setNotes("");
  };

  const startDeadlineEdit = (deadline) => {
    setEditingDeadlineId(deadline.id);
    setDeadlineDraft({
      title: deadline.title,
      subject: deadline.subject,
      dueDate: toDateInput(deadline.due_date),
      reminderDate: toDateInput(deadline.reminder_date),
      notes: deadline.notes || "",
    });
  };

  const cancelDeadlineEdit = () => {
    setEditingDeadlineId("");
  };

  const saveDeadlineEdit = async () => {
    if (!editingDeadlineId || !deadlineDraft.title.trim() || !deadlineDraft.dueDate) {
      return;
    }

    await onUpdateDeadline(editingDeadlineId, {
      title: deadlineDraft.title,
      subject: deadlineDraft.subject,
      due_date: new Date(deadlineDraft.dueDate).toISOString(),
      reminder_date: deadlineDraft.reminderDate ? new Date(deadlineDraft.reminderDate).toISOString() : null,
      notes: deadlineDraft.notes,
    });
    setEditingDeadlineId("");
  };

  return (
    <section className="space-y-8" data-testid="deadlines-page-root">
      <div className="grid grid-cols-1 gap-6 xl:grid-cols-2">
        <BrutalCard
```

```
 121|           className="bg-[#fed7aa]"
 122|           title="Add Deadline"
 123|           subtitle="Track exams, submissions, and milestones."
 124|           cardTestId="deadline-form-card"
 125|         >
 126|           <form className="space-y-5" onSubmit={submitDeadline} data-testid="deadline-form">
 127|             <div className="space-y-2">
 128|               <label htmlFor="deadline-title" className="text-sm font-medium"
data-testid="deadline-title-label">Title</label>
 129|               <Input
 130|                 id="deadline-title"
 131|                 value={title}
 132|                 onChange={(event) => setTitle(event.target.value)}
 133|                 required
 134|                 placeholder="Math final exam"
 135|                 className="border-2 border-[#18181b] rounded-none"
 136|                 data-testid="deadline-title-input"
 137|               />
 138|             </div>
 139|
 140|             <div className="space-y-2">
 141|               <label htmlFor="deadline-subject" className="text-sm font-medium"
data-testid="deadline-subject-label">Subject</label>
 142|               <Input
 143|                 id="deadline-subject"
 144|                 value={subject}
 145|                 onChange={(event) => setSubject(event.target.value)}
 146|                 className="border-2 border-[#18181b] rounded-none"
 147|                 data-testid="deadline-subject-input"
 148|               />
 149|             </div>
 150|
 151|             <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
 152|               <DatePickerField
 153|                 id="deadline-due-date"
 154|                 label="Due date"
 155|                 value={dueDate}
 156|                 onChange={setDueDate}
 157|                 required
 158|                 testIdPrefix="deadline-due-date"
 159|               />
 160|               <DatePickerField
 161|                 id="deadline-reminder-date"
 162|                 label="Reminder date"
 163|                 value={reminderDate}
 164|                 onChange={setReminderDate}
 165|                 testIdPrefix="deadline-reminder-date"
 166|               />
 167|             </div>
 168|
 169|             <div className="space-y-2">
 170|               <label htmlFor="deadline-notes" className="text-sm font-medium"
data-testid="deadline-notes-label">Notes</label>
 171|               <Textarea
 172|                 id="deadline-notes"
 173|                 value={notes}
 174|                 onChange={(event) => setNotes(event.target.value)}
 175|                 className="min-h-20 border-2 border-[#18181b] rounded-none"
 176|                 data-testid="deadline-notes-input"
 177|               />
 178|             </div>
 179|
 180|             <Button
 181|               type="submit"
 182|               disabled={submittingDeadline || !title.trim() || !dueDate}
 183|               className="w-full rounded-full border-2 border-[#18181b] bg-[#18181b] py-6 text-base font-bold text-white
transition-transform hover:scale-105"
 184|               data-testid="deadline-submit-button"
 185|             >
 186|               {submittingDeadline ? "Adding..." : "Save Deadline"}
 187|             </Button>
 188|           </form>
 189|         </BrutalCard>
 190|
 191|         <BrutalCard className="bg-white" title="Deadline Alerts" subtitle="Critical reminders from your timeline"
cardTestId="deadline-reminder-card">
 192|           <div className="space-y-3" data-testid="deadline-reminder-list">
 193|             {reminders.length === 0 ? (
 194|               <p className="text-sm" data-testid="deadline-reminder-empty">No urgent reminders now.</p>
 195|             ) : (
 196|               reminders.map((item, index) => (
 197|                 <div
 198|                   className={`flex items-center justify-between border-2 border-[#18181b] p-3
${urgencyColor(item.days_left)}`}
```

```
199|                  key={item.id}
200|                  data-testid={`deadline-reminder-item-${index}`}
201|                >
202|                  <div>
203|                    <p className="font-medium" data-testid={`deadline-reminder-item-${index}-title`}>{item.title}</p>
204|                    <p className="text-sm" data-testid={`deadline-reminder-item-${index}-days`}>
205|                      {item.days_left < 0 ? `${Math.abs(item.days_left)} days overdue` : `${item.days_left} days left`}
206|                    </p>
207|                  </div>
208|                  <Bell size={18} />
209|                </div>
210|              ))
211|            )}
212|          </div>
213|        </BrutalCard>
214|      </div>
215|
216|      <BrutalCard className="bg-[#bfdbfe]" title="All Deadlines" subtitle="Mark complete or remove as needed"
cardTestId="deadline-list-card">
217|        <div className="space-y-3" data-testid="deadline-list-wrap">
218|          {deadlines.length === 0 ? (
219|            <p className="text-sm" data-testid="deadline-empty-message">No deadlines yet.</p>
220|          ) : (
221|            deadlines.map((deadline, index) => (
222|              <motion.div
223|                key={deadline.id}
224|                custom={index}
225|                variants={DEADLINE_ITEM_VARIANTS}
226|                initial="hidden"
227|                animate="visible"
228|                className="flex flex-wrap items-center justify-between gap-3 border-b-2 border-[#18181b] pb-3"
229|                data-testid={`deadline-item-${index}`}
230|              >
231|                {editingDeadlineId === deadline.id ? (
232|                  <div className="w-full space-y-4" data-testid={`deadline-item-${index}-edit-wrap`}>
233|                    <div className="grid grid-cols-1 gap-3 md:grid-cols-2">
234|                      <div className="space-y-2">
235|                        <label className="text-sm font-medium" data-testid={`deadline-item-${index}-edit-title-label`}>
236|                          Title
237|                        </label>
238|                        <Input
239|                          value={deadlineDraft.title}
240|                          onChange={(event) => setDeadlineDraft((prev) => ({ ...prev, title: event.target.value }))}
241|                          className="border-2 border-[#18181b] rounded-none"
242|                          data-testid={`deadline-item-${index}-edit-title-input`}
243|                        />
244|                      </div>
245|                      <div className="space-y-2">
246|                        <label className="text-sm font-medium" data-testid={`deadline-item-${index}-edit-subject-label`}>
247|                          Subject
248|                        </label>
249|                        <Input
250|                          value={deadlineDraft.subject}
251|                          onChange={(event) => setDeadlineDraft((prev) => ({ ...prev, subject: event.target.value }))}
252|                          className="border-2 border-[#18181b] rounded-none"
253|                          data-testid={`deadline-item-${index}-edit-subject-input`}
254|                        />
255|                      </div>
256|                    </div>
257|                    <div className="grid grid-cols-1 gap-3 md:grid-cols-2">
258|                      <DatePickerField
259|                        id={`deadline-edit-due-date-${index}`}
260|                        label="Due date"
261|                        value={deadlineDraft.dueDate}
262|                        onChange={(value) => setDeadlineDraft((prev) => ({ ...prev, dueDate: value }))}
263|                        required
264|                        testIdPrefix={`deadline-item-${index}-edit-due-date`}
265|                      />
266|                      <DatePickerField
267|                        id={`deadline-edit-reminder-date-${index}`}
268|                        label="Reminder date"
269|                        value={deadlineDraft.reminderDate}
270|                        onChange={(value) => setDeadlineDraft((prev) => ({ ...prev, reminderDate: value }))}
271|                        testIdPrefix={`deadline-item-${index}-edit-reminder-date`}
272|                      />
273|                    </div>
274|                    <div className="space-y-2">
275|                      <label className="text-sm font-medium" data-testid={`deadline-item-${index}-edit-notes-label`}>
276|                        Notes
277|                      </label>
278|                      <Textarea
279|                        value={deadlineDraft.notes}
280|                        onChange={(event) => setDeadlineDraft((prev) => ({ ...prev, notes: event.target.value }))}
281|                        className="min-h-20 border-2 border-[#18181b] rounded-none"
```

```
                        data-testid={`deadline-item-${index}-edit-notes-input`}
                      />
                    </div>
                    <div className="flex flex-wrap gap-2">
                      <Button
                        type="button"
                        onClick={saveDeadlineEdit}
                        className="rounded-full border-2 border-[#18181b] bg-[#18181b] text-white hover:scale-105"
                        data-testid={`deadline-item-${index}-edit-save-button`}
                      >
                        <Save size={ACTION_ICON_SIZE} className="mr-1" />
                        Save
                      </Button>
                      <Button
                        type="button"
                        onClick={cancelDeadlineEdit}
                        className="rounded-full border-2 border-[#18181b] bg-white text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                        data-testid={`deadline-item-${index}-edit-cancel-button`}
                      >
                        <X size={ACTION_ICON_SIZE} className="mr-1" />
                        Cancel
                      </Button>
                    </div>
                  </div>
                ) : (
                  <>
                    <div>
                      <p className="font-medium" data-testid={`deadline-item-${index}-title`}>{deadline.title}</p>
                      <p className="text-sm" data-testid={`deadline-item-${index}-meta`}>
                        {deadline.subject} • Due {formatDate(deadline.due_date)}
                      </p>
                    </div>
                    <div className="flex gap-2">
                      <Button
                        type="button"
                        onClick={() => startDeadlineEdit(deadline)}
                        className="rounded-full border-2 border-[#18181b] bg-[#e9d5ff] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                        data-testid={`deadline-item-${index}-edit-button`}
                      >
                        <Pencil size={ACTION_ICON_SIZE} />
                      </Button>
                      <Button
                        type="button"
                        onClick={() => onToggleDeadline(deadline)}
                        className="rounded-full border-2 border-[#18181b] bg-[#bbf7d0] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                        data-testid={`deadline-item-${index}-toggle-button`}
                      >
                        <Check size={ACTION_ICON_SIZE} />
                      </Button>
                      <Button
                        type="button"
                        onClick={() => onDeleteDeadline(deadline.id)}
                        className="rounded-full border-2 border-[#18181b] bg-[#fecaca] text-[#18181b] hover:bg-[#18181b]
hover:text-white"
                        data-testid={`deadline-item-${index}-delete-button`}
                      >
                        <Trash2 size={ACTION_ICON_SIZE} />
                      </Button>
                    </div>
                  </>
                )}
              </motion.div>
            ))
          )}
        </div>
      </BrutalCard>
    </section>
  );
}
```

FILE: /app/frontend/src/pages/AnalyticsPage.jsx

```
import { motion } from "framer-motion";
import { BrutalCard } from "@/components/BrutalCard";

const metricCards = [
  { key: "tasks_total", label: "Total Tasks", bg: "bg-[#bfdbfe]" },
  { key: "tasks_completed", label: "Completed Tasks", bg: "bg-[#bbf7d0]" },
  { key: "focus_minutes_total", label: "Focus Minutes", bg: "bg-[#fecaca]" },
  { key: "productivity_score", label: "Productivity Score", bg: "bg-[#e9d5ff]", suffix: "%" },
];

```

```
const ANALYTICS_CARD_STAGGER = 0.05;
const ANALYTICS_CARD_VARIANTS = {
  hidden: { opacity: 0, y: 12 },
  visible: (index) => ({
    opacity: 1,
    y: 0,
    transition: { delay: index * ANALYTICS_CARD_STAGGER },
  }),
};

export default function AnalyticsPage({ analytics }) {
  const completionRate = analytics.tasks_total
    ? Math.round((analytics.tasks_completed / analytics.tasks_total) * 100)
    : 0;

  return (
    <section className="space-y-8" data-testid="analytics-page-root">
      <div className="grid grid-cols-1 gap-6 md:grid-cols-2 xl:grid-cols-4" data-testid="analytics-metrics-grid">
        {metricCards.map((metric, index) => (
          <motion.div
            key={metric.key}
            custom={index}
            variants={ANALYTICS_CARD_VARIANTS}
            initial="hidden"
            animate="visible"
          >
            <BrutalCard
              className={metric.bg}
              title={metric.label}
              subtitle="Live planner data"
              cardTestId={`analytics-metric-${metric.key}-card`}
            >
              <p className="text-3xl font-semibold md:text-4xl" data-testid={`analytics-metric-${metric.key}-value`}>
                {analytics[metric.key]}
                {metric.suffix || ""}
              </p>
            </BrutalCard>
          </motion.div>
        ))}
      </div>

      <div className="grid grid-cols-1 gap-6 xl:grid-cols-2">
        <BrutalCard className="bg-white" title="Completion Trend" subtitle="Task completion quality"
cardTestId="analytics-completion-card">
          <div className="space-y-4" data-testid="analytics-completion-wrap">
            <p className="text-sm md:text-base" data-testid="analytics-completion-rate-text">
              You are currently completing <strong>{completionRate}%</strong> of your tasks.
            </p>
            <div className="h-4 w-full border-2 border-[#18181b] bg-[#f4f4f5]" data-testid="analytics-completion-track">
              <div
                className="h-full bg-[#18181b] transition-[width] duration-300"
                style={{ width: `${completionRate}%` }}
                data-testid="analytics-completion-fill"
              />
            </div>
            <p className="text-sm" data-testid="analytics-upcoming-deadlines-text">
              Upcoming active deadlines: <strong>{analytics.deadlines_upcoming}</strong>
            </p>
          </div>
        </BrutalCard>

        <BrutalCard className="bg-[#bbf7d0]" title="Subject Distribution" subtitle="Where your effort is going"
cardTestId="analytics-subject-card">
          <div className="space-y-4" data-testid="analytics-subject-list">
            {analytics.subject_breakdown.length === 0 ? (
              <p className="text-sm" data-testid="analytics-subject-empty">No subject data yet. Add tasks to see
insights.</p>
            ) : (
              analytics.subject_breakdown.map((item, index) => {
                const ratio = item.total_tasks ? Math.round((item.completed_tasks / item.total_tasks) * 100) : 0;
                return (
                  <div key={`${item.subject}-${index}`} data-testid={`analytics-subject-item-${index}`}>
                    <div className="mb-1 flex items-center justify-between text-sm">
                      <span data-testid={`analytics-subject-item-${index}-name`}>{item.subject}</span>
                      <span data-testid={`analytics-subject-item-${index}-ratio`}>{ratio}% completed</span>
                    </div>
                    <div className="h-3 w-full border-2 border-[#18181b] bg-white"
data-testid={`analytics-subject-item-${index}-track`}>
                      <div
                        className="h-full bg-[#18181b] transition-[width] duration-300"
                        style={{ width: `${ratio}%` }}
                        data-testid={`analytics-subject-item-${index}-fill`}
                      />
                    </div>
```

```
                  </div>
                );
              })
            )}
          </div>
        </BrutalCard>
      </div>
    </section>
  );
}
```